
IoT-Based LoRa Announcement System

Cloud-managed, low-power campus tutorial-room signage

Team Clankers

Youssef Ahmed Hussein Abdelaziz
Ahmed Hazem
Mohamed Waleed
Youssef El Rayes

Implemented System

ThingsBoard Cloud + MQTT gateway
Addressed LoRa delivery
Two low-power e-paper receivers

Problem Statement

Current Communication Problem

- ▶ Tutorial updates are distributed through emails, verbal announcements, or manual boards.
- ▶ Urgent room changes and cancellations can be missed.
- ▶ Students may arrive at the wrong room.
- ▶ Teaching staff repeatedly communicate the same update.

Engineering Need

- ▶ Remotely managed room signage
- ▶ Long-range delivery without WiFi at every display
- ▶ Persistent last-known announcement
- ▶ Emergency updates to multiple rooms
- ▶ Reduced receiver power consumption

Related Work and Identified Gap

Smart-campus IoT architectures

- ▶ Central cloud platform manages connected devices.
- ▶ Supports monitoring and smart-classroom services.
- ▶ Often depends on WiFi coverage at every endpoint.

Internet-connected notice boards

- ▶ Allow remote updates from web or mobile interfaces.
- ▶ Usually require every display to maintain an internet connection.

Gap Addressed by Our System

Separate cloud connectivity from the classroom “last mile”:

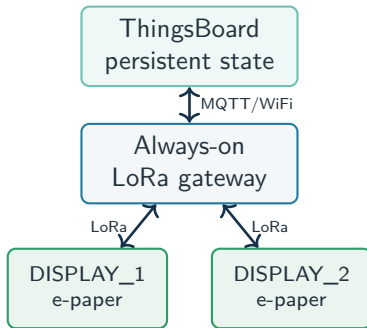
- ▶ One always-connected gateway uses WiFi and MQTT.
- ▶ Classroom receivers use long-range LoRa.
- ▶ E-paper keeps showing the last state without continuous refresh.
- ▶ Receivers poll on a low-power schedule.

Proposed Solution

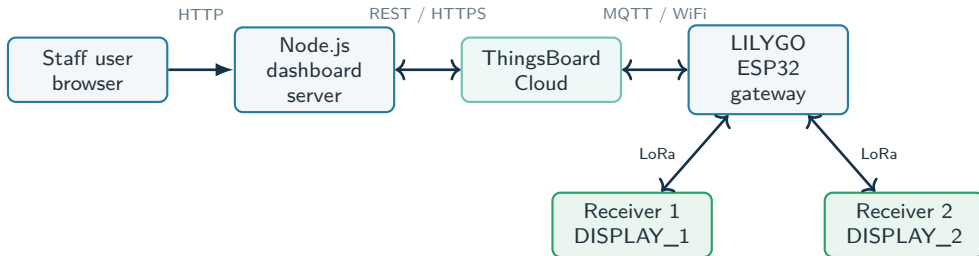
Core Idea

Staff submit tutorial announcements through a web dashboard. ThingsBoard stores the desired state for each room. An ESP32 gateway receives cloud updates and serves them to addressed LoRa receivers.

- ▶ Normal message: target one display
- ▶ Emergency message: update both displays
- ▶ Version number: prevent unnecessary e-paper refresh
- ▶ ACK telemetry: confirm physical delivery



System Architecture



ThingsBoard shared attributes

Gateway cache

ThingsBoard telemetry

Desired display message and version

Latest state available for each receiver poll

Applied version, receiver RSSI, and online status

ThingsBoard Cloud Operation

Dashboard Server: REST API

- ▶ Authenticates using JWT or tenant credentials.
- ▶ Finds DISPLAY_1 and DISPLAY_2.
- ▶ Reads and writes shared attributes.
- ▶ Creates increasing message versions.
- ▶ Republishes stored state for gateway recovery.

ESP32 Gateway: MQTT API

- ▶ Authenticates using a gateway access token.
- ▶ Subscribes to v1/gateway/attributes.
- ▶ Caches incoming display states.
- ▶ Publishes ACK data to v1/gateway/telemetry.

Desired state flows down; observed delivery telemetry flows back up.

Gateway Operation

Always-on responsibilities

- ▶ Maintain WiFi and MQTT connections
- ▶ Receive ThingsBoard shared-attribute updates
- ▶ Cache the latest state of both displays
- ▶ Continuously listen for LoRa requests
- ▶ Return the requested display state
- ▶ Forward ACK information to ThingsBoard

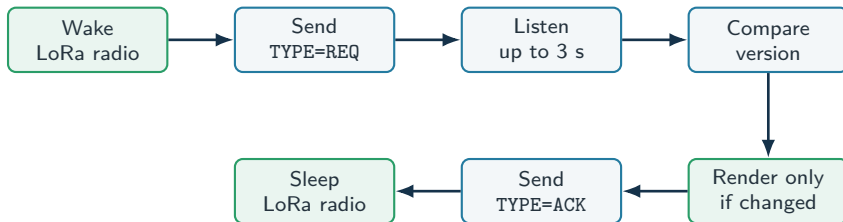
Important Design Change

ThingsBoard updates no longer trigger immediate LoRa transmissions. They only update the gateway cache.

Reason

Receivers can sleep their LoRa radios and request data only at scheduled poll times.

Low-Power Receiver Polling



Receiver 1

Polls every 60 seconds as `DISPLAY_1`.

Receiver 2

Uses the same 60-second cadence, offset by about 30 seconds.

LoRa Packet Protocol

Receiver requests its state

```
TYPE=REQ|SOURCE=DISPLAY_1|VERSION=11
```

Gateway returns cached ThingsBoard state

```
TYPE=DATA|TARGET=DISPLAY_1|PRIORITY=NORMAL|TUT=T01|COURSE=IoT|  
SLOT=10:00-11:00|ROOM=C7.201|MSG=Class starts soon.|VERSION=12
```

Receiver confirms delivery

```
TYPE=ACK|SOURCE=DISPLAY_1|VERSION=12|RSSI=-74
```

Addressing prevents messages from appearing on the wrong room display.

Implementation Stack

Layer	Technology	Responsibility
Web interface	HTML, CSS, browser JavaScript	Message entry, target selection, emergency action
Dashboard back-end	Node.js 18+	Validation and ThingsBoard REST calls
Cloud	ThingsBoard Cloud	Persistent shared attributes and telemetry
Gateway firmware	ESP32 Arduino C++, WiFi, PubSubClient, ArduinoJson	MQTT bridge, state cache, LoRa request handling
Receiver firmware	ESP32 Arduino C++, LoRa, Gx-EPD2	Polling, version checks, ACKs, e-paper rendering
Hardware	3 LILYGO LoRa32 T3 boards, 2 Waveshare 4.2-inch displays	Gateway and classroom endpoints

Progress Achieved

Hardware and LoRa

- ✓ Wired and configured Waveshare e-paper displays
- ✓ Replaced OLED output with e-paper rendering
- ✓ Established point-to-point LoRa communication
- ✓ Defined addressed packet format
- ✓ Added two independent display receivers

Cloud and Application

- ✓ Built web dashboard and Send action
- ✓ Integrated ThingsBoard REST and MQTT
- ✓ Added emergency messaging
- ✓ Added message priority and version handling
- ✓ Added ACK telemetry and RSSI reporting

Completed Scope from the Project Plan

Completed Red-Marked Tasks

- ▶ E-paper wiring, configuration, rendering, and testing
- ▶ Point-to-point LoRa communication and packet format
- ▶ Web dashboard with message input and Send action
- ▶ Dashboard/cloud messaging integration
- ▶ Emergency priority announcements
- ▶ Multi-node addressing and priority handling
- ▶ Network-accessible dashboard for multiple devices

Implementation Notes

- ▶ Cloud integration uses ThingsBoard REST + MQTT instead of HiveMQ WebSockets.
- ▶ Multi-node support includes two receivers, versions, addressing, and ACK telemetry.
- ▶ Browser dashboard provides mobile-device access without a native app.

Remaining Validation

LoRa radio sleeping is implemented.

Quantitative power-consumption

12/16

End-to-End Demonstration Flow

1. Staff submits a normal message for `DISPLAY_1`.
2. Dashboard server writes new shared attributes and a new version to ThingsBoard.
3. Gateway receives the MQTT update and caches it without transmitting immediately.
4. Receiver 1 wakes at its next poll and sends a LoRa request.
5. Gateway sends the cached state; Receiver 2 ignores traffic not addressed to it.
6. Receiver 1 refreshes the e-paper only because the version changed.
7. Receiver 1 sends an ACK; gateway publishes its version and RSSI as telemetry.
8. An emergency message updates both displays at their respective next polls.

Key Design Decisions

Pull-Based Delivery

Receivers request data rather than remaining in continuous LoRa receive mode.

Persistent Cloud State

ThingsBoard shared attributes retain the latest desired message.

Version-Based Rendering

Repeated state does not trigger another e-paper refresh.

Staggered Polling

Receiver polls are offset to reduce LoRa collisions.

Always-On Gateway

The gateway remains available for MQTT updates and receiver requests.

Protected Credentials

Real ThingsBoard tokens and dashboard credentials remain in ignored local files.

Challenges, Limitations, and Future Work

Current limitations

- ▶ Delivery latency can reach one polling interval.
- ▶ Receiver ESP32 remains awake; only its LoRa radio sleeps.
- ▶ Gateway cache is stored in RAM.
- ▶ LoRa application packets are not encrypted.
- ▶ Current firmware targets two fixed display IDs.

Next steps

- ▶ Measure receiver power consumption before and after radio sleeping.
- ▶ Add ESP32 deep sleep where practical.
- ▶ Test range and packet reliability across campus buildings.
- ▶ Add scheduling and timetable synchronization.
- ▶ Add application-layer encryption and stronger delivery retry logic.
- ▶ Scale device registration beyond two displays.

Conclusion

Result

The project now provides an end-to-end announcement pipeline from a web dashboard, through ThingsBoard Cloud and an MQTT/LoRa gateway, to two addressed e-paper receivers.

Cloud-managed

Persistent desired state

Low-power

Scheduled radio polling

Observable

ACK and RSSI telemetry

Thank you